Take-home challenge: Finance spend intelligence assistant

Time box: ~3–4 hours. Please don't go over — we'd rather see a focused, well-reasoned prototype than something exhaustive. We'll cover the rest in the conversation.

Next step: During the technical interview, you'll walk us through what you built and why.

What this is about

Our Finance team spends a lot of time cross-referencing vendor invoices with our procurement policy — checking approvals, vendor status, spend thresholds. We want to see how you'd build a simple AI assistant to help with that.

You're not being asked to solve the whole problem. We want to see how you think about LLM system design, how you wire up a backend and frontend, and how you explain your decisions.

What's in the pack

File	What it is
`invoices_2025_clean.csv`	140 invoices from 2025. The data is clean — consistent dates, no duplicates. Some invoices are missing a PO number or approver; some vendors aren't active. That's intentional — it's realistic, not an error.
`vendor_master.csv`	10 vendors with their country, category, and status (ACTIVE / INACTIVE / ON_HOLD).
`procurement_policy.md`	A short excerpt of our Global Procurement & Spend Policy.

What we'd like you to build

A small web app — FastAPI backend, React frontend — where a finance analyst can ask questions about the invoice data and the policy.

Questions like:

- *"What are our top 5 vendors by total spend in 2025?"*
- *"What is our total spend with vendors based in Spain?"*
- *"Who is allowed to approve an invoice of EUR 60,000?"*

Backend

Two endpoints, minimum:

- `POST /ask` — takes a question, returns an answer and the evidence behind it (which part of the policy, which rows from the data)
- `GET /summary` — returns a quick overview of the invoice data (total spend, breakdown by currency and vendor status)

Frontend

- A chat panel where the analyst types questions and sees answers with their supporting evidence
- A summary panel that loads on startup from `GET /summary` — there should be something on screen before the first question is asked

We're not looking at visual design. We're looking at how you handle state, errors, loading, and — most importantly — how the UI makes it clear *what the answer is based on*.

On the LLM

Use whatever you have access to. If you don't have an API key, a clearly labelled stub is fine — we're evaluating the architecture and prompts, not the model's prose. Ollama works too. Just note in your README which path you took.

The policy doc is about 50 lines — load it straight into context. You don't need a vector database here. If you build one anyway, explain the reasoning.

A short write-up (half a page in your README)

Three things:

1. **What you built and what you left out** — and why. We want to understand your priorities.
2. **How you'd evaluate it** — not asking you to build an eval suite, just describe concretely how you'd measure answer quality and catch hallucinations.
3. **What production would need** — think about 200 Finance users, real data, audit trails (policy §8.2), access control, cost.

If you have time left

Pick one at most:

- A small eval set (5–10 question/answer pairs) with a scoring script
- Guardrails — the assistant declines or escalates when it can't answer from available data
- Agentic routing — the assistant reasons explicitly about whether to hit the database, the policy, or both, and tells the user
- Chat history — persist conversations to the database and load them on return visits

---

Deliverables

Please share a **public GitHub repo** at least 48 hours before your technical interview. Include:

- Working backend and frontend with setup instructions in the README
- The write-up above in the README
- A quick note on how long it took and what you'd do next

Docker Compose is welcome but not expected.

A few ground rules

- Python + FastAPI and React please. If you go a different direction, just say why.
- Any libraries are fair game.
- **AI coding tools are fine** — but be ready to explain every part of what you submit. If you used AI tools to build this, we'd like to see your prompts and any AI harness included in the repo. How you work with AI is part of what we're evaluating.
- Don't spend time on auth, deployment, CI/CD, or making it look polished.

What we're looking at

Area	What we care about
LLM system design	How you construct prompts, handle grounding, route between policy and data, deal with failure
Full-stack design	API contract, state management, error and loading states, how evidence is surfaced in the UI
Code quality	Clear, readable code appropriate for a prototype
Written reasoning	How you explain decisions and trade-offs

Good luck — looking forward to the conversation.